

Vector vs. diccionario para mapas pequeños

Eduardo Alanis García

2026-08-26

Table of contents

1	Actividad: Vector vs. diccionario para mapas pequeños	2
1.1	Introducción	2
1.2	Implementación	2
1.2.1	Valor centinela	2
1.2.2	Vector de capacidad fija	2
1.2.3	Implementación de buscar para el vector	2
1.2.4	Implementación de insertar! para el vector	3
1.2.5	Implementación de borrar! para el vector	4
1.3	Implementación del diccionario	4
1.4	Preparación de los datos	5
1.5	Comprobación básica de las funciones	6
2	Experimentos	6
2.1	Tamaños de entrada	6
2.2	Experimento: buscar	7
2.2.1	Resultados de buscar	8
2.2.2	Análisis de buscar	10
2.3	Experimento: borrar!	11
2.3.1	Resultados de borrar!	12
2.3.2	Análisis de borrar!	14
2.4	Experimento: insertar!	14
2.4.1	Resultados de insertar!	15
2.4.2	Análisis de insertar!	17
3	Comparación de rangos	18
4	Discusión	19
5	Conclusiones	19

1 Actividad: Vector vs. diccionario para mapas pequeños

1.1 Introducción

En esta actividad se compara el desempeño de dos estructuras de datos para representar un mapa de llaves enteras (`Int64`) a valores flotantes (`Float64`). La primera estructura es un vector de pares sin ordenar, en el que las operaciones se realizan mediante un recorrido lineal. La segunda estructura es un `Dict{Int64,Float64}`, que utiliza una tabla hash.

El objetivo es comparar experimentalmente las operaciones `buscar`, `insertar!` y `borrar!` para diferentes tamaños de entrada. De esta manera, los resultados experimentales permiten observar el comportamiento de cada estructura en una computadora real.

1.2 Implementación

1.2.1 Valor centinela

Se utilizará una llave que no forme parte de los datos reales:

```
CENTINELA = typemin{Int64}
```

-9223372036854775808

1.2.2 Vector de capacidad fija

El vector se crea con una capacidad determinada desde el inicio. Esto permite mantener fija su longitud durante las operaciones y evita utilizar `push!`, `pop!` o `deleteat!`.

```
function crear_vector(capacidad)
    return Vector{Pair{Int64,Float64}}([CENTINELA => 0.0 for i in 1:capacidad])
end
```

`crear_vector` (generic function with 1 method)

1.2.3 Implementación de buscar para el vector

```

function buscar_vector(vector, llave)
    for par in vector
        if par.first != CENTINELA && par.first == llave
            return par.second
        end
    end
    return nothing
end

```

buscar_vector (generic function with 1 method)

Si la llave no se encuentra, devuelve nothing.

1.2.4 Implementación de insertar! para el vector

```

function insertar_vector!(v, llave, valor)
    for i in 1:length(v)

        # Si la llave ya existe, se actualiza el valor
        if v[i].first == llave
            v[i] = llave => valor
            return v
        end

        # Si no existe, busca una posición marcada como vacía para colocar el nuevo par
        if v[i].first == CENTINELA
            v[i] = llave => valor
            return v
        end
    end

    error("No hay ningun espacio disponible en vector")
end

```

insertar_vector! (generic function with 1 method)

Si la llave ya existe, actualiza su valor. Si no existe, utiliza la primera posición marcada como vacía.

1.2.5 Implementación de borrar! para el vector

```
function borrar_vector!(v, llave)
    for i in 1:length(v)
        if v[i].first == llave
            v[i] = CENTINELA => 0.0
            return v
        end
    end

    return v
end
```

borrar_vector! (generic function with 1 method)

La posición encontrada no se elimina. En su lugar, se sobrescribe con el centinela para indicar que está disponible nuevamente.

1.3 Implementación del diccionario

Funciones equivalentes para poder comparar ambas estructuras.

```
function crear_dict_mapa(capacidad::Int)
    d = Dict{Int64,Float64}()
    sizehint!(d, capacidad)
    return d
end

function buscar_dict(d, llave)
    if haskey(d, llave)
        return d[llave]
    end
    return nothing
end

function insertar_dict!(d, llave, valor)
    d[llave] = valor
    return d
end
```

```
function borrar_dict!(d, llave)
    delete!(d, llave)
    return d
end
```

borrar_dict! (generic function with 1 method)

En el caso del Dict, `sizehint!` se utiliza para reservar espacio y reducir el efecto del crecimiento de la tabla durante los experimentos.

1.4 Preparación de los datos

Para cada tamaño `n` se crean ambas estructuras con las mismas llaves y los mismos valores. Las llaves se mezclan para capturar el caso promedio y no solo el mejor o peor caso posicional.

```
using BenchmarkTools
using Random
using DataFrames
using Plots
using Statistics

#Random.seed!(1234)

function preparar_datos(n, capacidad_vector = n)
    v = crear_vector(capacidad_vector)
    d = crear_dict_mapa(n)
    llaves = collect{Int64}(1:n)
    shuffle!(llaves) # Desordena de forma aleatoria, por la indicacion la llave a con
    valores = rand(n)

    for i in 1:n
        insertar_vector!(v, llaves[i], valores[i])
        insertar_dict!(d, llaves[i], valores[i])
    end

    return v, d
end
```

preparar_datos (generic function with 2 methods)

1.5 Comprobación básica de las funciones

Solo vemos que funcionan correctamente.

```
v_prueba, d_prueba = preparar_datos(8, 9)
llave_prueba = 3

println("Resultado del vector: ", buscar_vector(v_prueba, llave_prueba))
println("Resultado del diccionario: ", buscar_dict(d_prueba, llave_prueba))
```

Resultado del vector: 0.15201743379947397
Resultado del diccionario: 0.15201743379947397

```
insertar_vector!(v_prueba, 9, 10.0)
insertar_dict!(d_prueba, 9, 10.0)

println("Resultado del vector: ", buscar_vector(v_prueba, 9))
println("Resultado del diccionario: ", buscar_dict(d_prueba, 9))
```

Resultado del vector: 10.0
Resultado del diccionario: 10.0

```
borrar_vector!(v_prueba, 9)
borrar_dict!(d_prueba, 9)

println("Resultado del vector: ", buscar_vector(v_prueba, 9))
println("Resultado del diccionario: ", buscar_dict(d_prueba, 9))
```

Resultado del vector: nothing
Resultado del diccionario: nothing

2 Experimentos

2.1 Tamaños de entrada

Se utilizarán potencias de 2 comenzando en 2^2 . Vamos a considera $2^{12} = 4096$ como máximo práctico dado que no tarda tanto mi PC.

```
tamaños = 2 .^ (2:12)
tamaños
```

```
11-element Vector{Int64}:
 4
 8
16
32
64
128
256
512
1024
2048
4096
```

2.2 Experimento: buscar

La llave se elige aleatoriamente dentro del `setup=` para cada muestra. De esta manera, no se utiliza siempre la misma posición del vector.

```
resultados_buscar_vector = []
resultados_buscar_dict = []

for n in tamaños
    v, d = preparar_datos(n)

    resultado_vector = @benchmark buscar_vector($v, llave) setup=(
        llave = rand{Int64}(1:$n)
    ) samples=100

    resultado_dict = @benchmark buscar_dict($d, llave) setup=(
        llave = rand{Int64}(1:$n)
    ) samples=100

    push!(resultados_buscar_vector,
        (
            n = n,
            trial = resultado_vector,
            minimo = minimum(resultado_vector).time,
            medio = mean(resultado_vector).time,
```

```

        maximo = maximum(resultado_vector).time
    )
)

push!(resultados_buscar_dict,
    (
        n = n,
        trial = resultado_dict,
        minimo = minimum(resultado_dict).time,
        medio = mean(resultado_dict).time,
        maximo = maximum(resultado_dict).time
    )
)
end

```

2.2.1 Resultados de buscar

```

tabla_buscar = DataFrame(
    n = [r.n for r in resultados_buscar_vector],
    vector_minimo = [r.minimo for r in resultados_buscar_vector],
    vector_medio = [r.medio for r in resultados_buscar_vector],
    vector_maximo = [r.maximo for r in resultados_buscar_vector],
    dict_minimo = [r.minimo for r in resultados_buscar_dict],
    dict_medio = [r.medio for r in resultados_buscar_dict],
    dict_maximo = [r.maximo for r in resultados_buscar_dict]
)

tabla_buscar.comparar = ifelse.(
    tabla_buscar.vector_medio .< tabla_buscar.dict_medio,
    "Vector",
    "Dict"
)

tabla_buscar

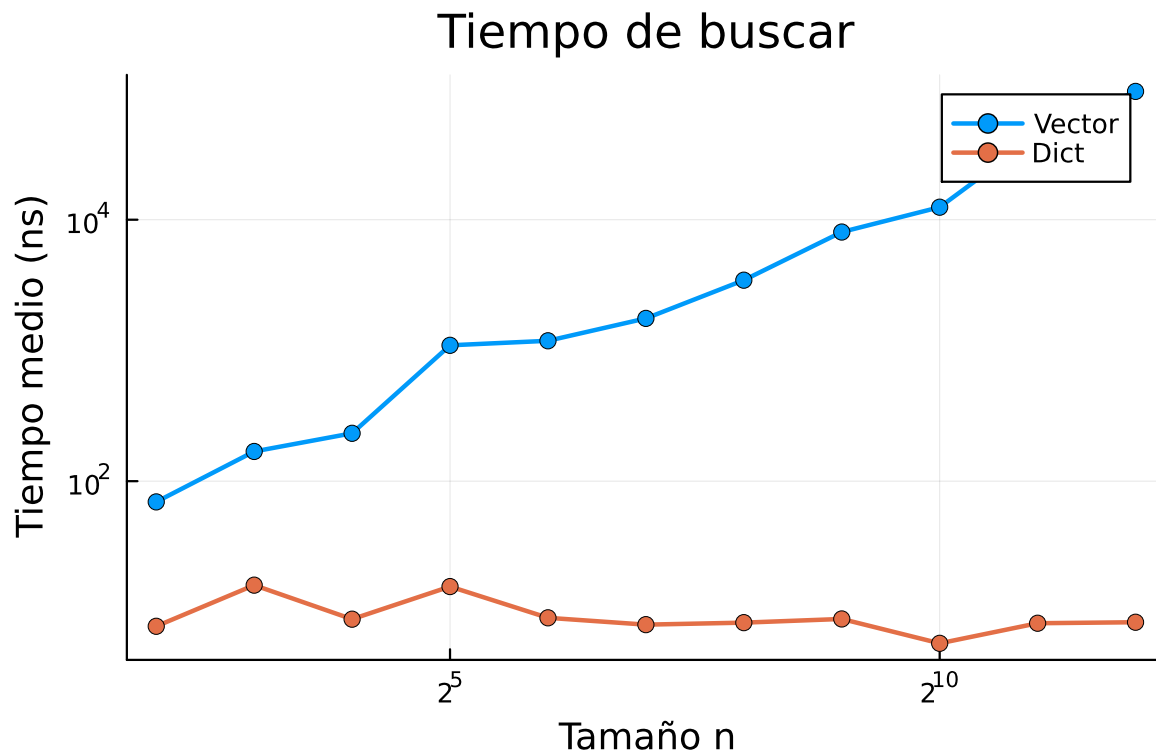
```

	n	vector_minimo	vector_medio	vector_maximo	dict_minimo	dict_medio	dict_maximo
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	4	29.7487	69.4241	112.864	7.2	7.762	21.1
2	8	30.4829	168.78	516.097	13.6136	16.023	140.541
3	16	30.4829	232.447	609.054	7.2	8.797	32.0
4	32	66.8342	1094.68	3708.74	7.91583	15.6253	34.3687
5	64	31.2877	1185.25	3553.82	7.2	9.014	154.8
6	128	130.986	1758.65	4784.81	7.2	7.99	13.5
7	256	61.167	3446.32	7525.05	7.2	8.278	16.8
8	512	32.5956	8049.14	23572.6	7.4	8.833	50.3
9	1024	1136.08	12466.6	32650.6	5.0	5.75	22.1
10	2048	117.224	44546.8	1.32099e5	7.2	8.193	29.4
11	4096	2360.37	95838.6	252012.0	7.2	8.337	16.1

```
ENV["GKSwstype"] = "100"
using Plots

default(fmt = :png)
```

```
plot(
    tabla_buscar.n,
    [tabla_buscar.vector_medio tabla_buscar.dict_medio],
    label=["Vector" "Dict"],
    xlabel="Tamaño n",
    ylabel="Tiempo medio (ns)",
    title="Tiempo de buscar",
    marker=:circle,
    linewidth=2,
    xscale=:log2,
    yscale=:log10
)
```



```
for fila in eachrow(tabla_buscar)
    println("n=$(fila.n): gana $(fila.comparar)")
end
```

```
n=4: gana Dict
n=8: gana Dict
n=16: gana Dict
n=32: gana Dict
n=64: gana Dict
n=128: gana Dict
n=256: gana Dict
n=512: gana Dict
n=1024: gana Dict
n=2048: gana Dict
n=4096: gana Dict
```

2.2.2 Análisis de buscar

Los resultados obtenidos muestran que el Dict presenta un mejor desempeño que el vector para la operación buscar en todos los tamaños de entrada evaluados. El tiempo medio del vector

crece de forma sostenida conforme aumenta n , mientras que el tiempo medio del diccionario se mantiene relativamente constante. Esto se debe a que la operación de búsqueda en un vector recorre todos los elementos hasta encontrar la llave, mientras que el diccionario utiliza una tabla hash que permite acceder directamente al valor asociado a la llave.

2.3 Experimento: borrar!

Como `borrar!` modifica la estructura, cada vez realizamos una copia de los datos. Además, se utiliza `evals=1` para que cada muestra del benchmark se ejecute una sola vez.

```
resultados_borrar_vector = []
resultados_borrar_dict = []

for n in tamaños
    v, d = preparar_datos(n)

    resultado_vector = @benchmark borrar_vector!(v, llave) setup=(
        v = copy($v);
        llave = rand{Int64}(1:$n)
    ) samples=100 evals=1

    resultado_dict = @benchmark borrar_dict!(d, llave) setup=(
        d = copy($d);
        llave = rand{Int64}(1:$n)
    ) samples=100 evals=1

    push!(resultados_borrar_vector, (
        n = n,
        trial = resultado_vector,
        minimo = minimum(resultado_vector).time,
        medio = mean(resultado_vector).time,
        maximo = maximum(resultado_vector).time
    ))

    push!(resultados_borrar_dict, (
        n = n,
        trial = resultado_dict,
        minimo = minimum(resultado_dict).time,
        medio = mean(resultado_dict).time,
        maximo = maximum(resultado_dict).time
    ))
end
```

2.3.1 Resultados de borrar!

```

tabla_borrar = DataFrame(
    n = [r.n for r in resultados_borrar_vector],
    vector_minimo = [r.minimo for r in resultados_borrar_vector],
    vector_medio = [r.medio for r in resultados_borrar_vector],
    vector_maximo = [r.maximo for r in resultados_borrar_vector],
    dict_minimo = [r.minimo for r in resultados_borrar_dict],
    dict_medio = [r.medio for r in resultados_borrar_dict],
    dict_maximo = [r.maximo for r in resultados_borrar_dict]
)

tabla_borrar.comparar = ifelse.(
    tabla_borrar.vector_medio .< tabla_borrar.dict_medio,
    "Vector",
    "Dict"
)

tabla_borrar

```

	n	vector_minimo	vector_medio	vector_maximo	dict_minimo	dict_medio	dict_maximo
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	4	200.0	350.0	6800.0	0.001	46.0005	100.0
2	8	200.0	354.0	6400.0	0.001	56.0004	100.0
3	16	200.0	339.0	5500.0	0.001	53.0005	100.0
4	32	200.0	343.0	5200.0	0.001	62.0004	100.0
5	64	200.0	323.0	5300.0	0.001	51.0005	100.0
6	128	400.0	569.0	7300.0	0.001	93.0001	200.0
7	256	200.0	605.0	17000.0	0.001	45.0006	100.0
8	512	300.0	527.0	6000.0	0.001	75.0003	200.0
9	1024	500.0	1733.0	42600.0	0.001	364.0	23600.0
10	2048	300.0	936.0	6600.0	0.001	91.0002	700.0
11	4096	400.0	1666.0	8000.0	0.001	92.0002	300.0

```

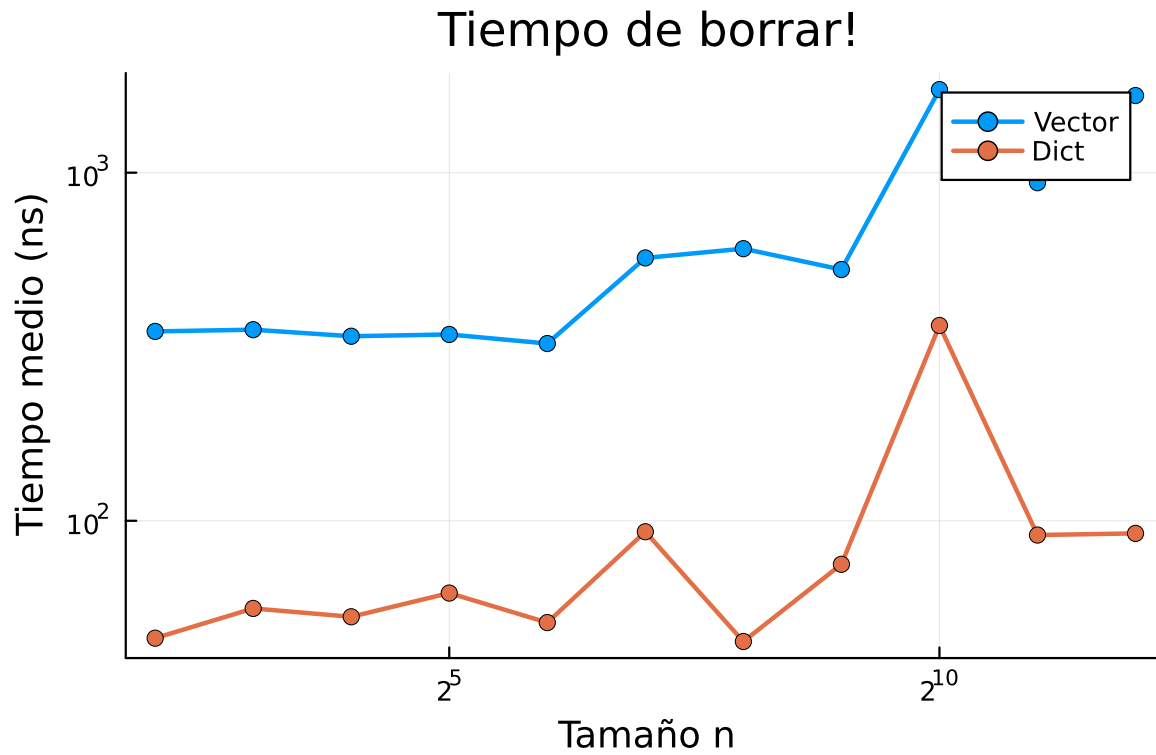
plot(
    tabla_borrar.n,
    [tabla_borrar.vector_medio tabla_borrar.dict_medio],
    label=["Vector" "Dict"],
    xlabel="Tamaño n",
    ylabel="Tiempo medio (ns)",
    title="Tiempo de borrar!",
    marker=:circle,

```

```

linewidth=2,
xscale=:log2,
yscale=:log10
)

```



```

for fila in eachrow(tabla_borrar)
    println("n=$(fila.n): gana $(fila.comparar)")
end

```

```

n=4: gana Dict
n=8: gana Dict
n=16: gana Dict
n=32: gana Dict
n=64: gana Dict
n=128: gana Dict
n=256: gana Dict
n=512: gana Dict
n=1024: gana Dict
n=2048: gana Dict
n=4096: gana Dict

```

2.3.2 Análisis de borrar!

Al igual que en **buscar**, la tabla y la gráfica de **borrar!** muestran que **Dict** es más rápido que el vector para todos los tamaños probados (**n** de 4 a 4096). El tiempo medio del vector crece de forma sostenida conforme aumenta **n** y es aproximadamente 10 veces mayor que el tiempo medio del diccionario para los tamaños más grandes. Esto se debe a que la operación de búsqueda en un vector sin ordenar requiere recorrer todos los elementos hasta encontrar la llave, mientras que el diccionario utiliza una tabla hash que permite acceder directamente al valor asociado a la llave. Además es interesante notar que el tiempo medio del diccionario se mantiene relativamente constante a medida que aumenta **n**, lo que indica que la complejidad de la operación de borrado es aproximadamente $O(1)$ en promedio, mientras que la complejidad del vector es $O(n)$.

2.4 Experimento: insertar!

Para la inserción se utiliza la llave **n + 1**, que no se encuentra entre las llaves iniciales en el caso del diccionario. En el caso del vector, se utiliza una capacidad de **n** dado que fue la instrucción de la actividad. Esto permite que la inserción se realice en la primera posición vacía.

```
resultados_insertar_vector = []
resultados_insertar_dict = []

for n in tamaños
    v, d = preparar_datos(n, n + 1)

    sizehint!(d, n + 1) # Para el diccionario

    resultado_vector = @benchmark insertar_vector!(v, nueva_llave, 1.0) setup=(
        v = copy($v);
        nueva_llave = Int64($n + 1)
    ) samples=100 evals=1

    resultado_dict = @benchmark insertar_dict!(d, nueva_llave, 1.0) setup=(
        d = copy($d);
        nueva_llave = Int64($n + 1)
    ) samples=100 evals=1

    push!(resultados_insertar_vector,
        (
            n = n,
            trial = resultado_vector,
```

```

        minimo = minimum(resultado_vector).time,
        medio = mean(resultado_vector).time,
        maximo = maximum(resultado_vector).time
    )
)

push!(resultados_insertar_dict,
(
    n = n,
    trial = resultado_dict,
    minimo = minimum(resultado_dict).time,
    medio = mean(resultado_dict).time,
    maximo = maximum(resultado_dict).time
)
)
end

```

2.4.1 Resultados de insertar!

```

tabla_insertar = DataFrame(
    n = [r.n for r in resultados_insertar_vector],
    vector_minimo = [r.minimo for r in resultados_insertar_vector],
    vector_medio = [r.medio for r in resultados_insertar_vector],
    vector_maximo = [r.maximo for r in resultados_insertar_vector],
    dict_minimo = [r.minimo for r in resultados_insertar_dict],
    dict_medio = [r.medio for r in resultados_insertar_dict],
    dict_maximo = [r.maximo for r in resultados_insertar_dict]
)

tabla_insertar.comparar = ifelse.(
    tabla_insertar.vector_medio .< tabla_insertar.dict_medio,
    "Vector",
    "Dict"
)

tabla_insertar

```

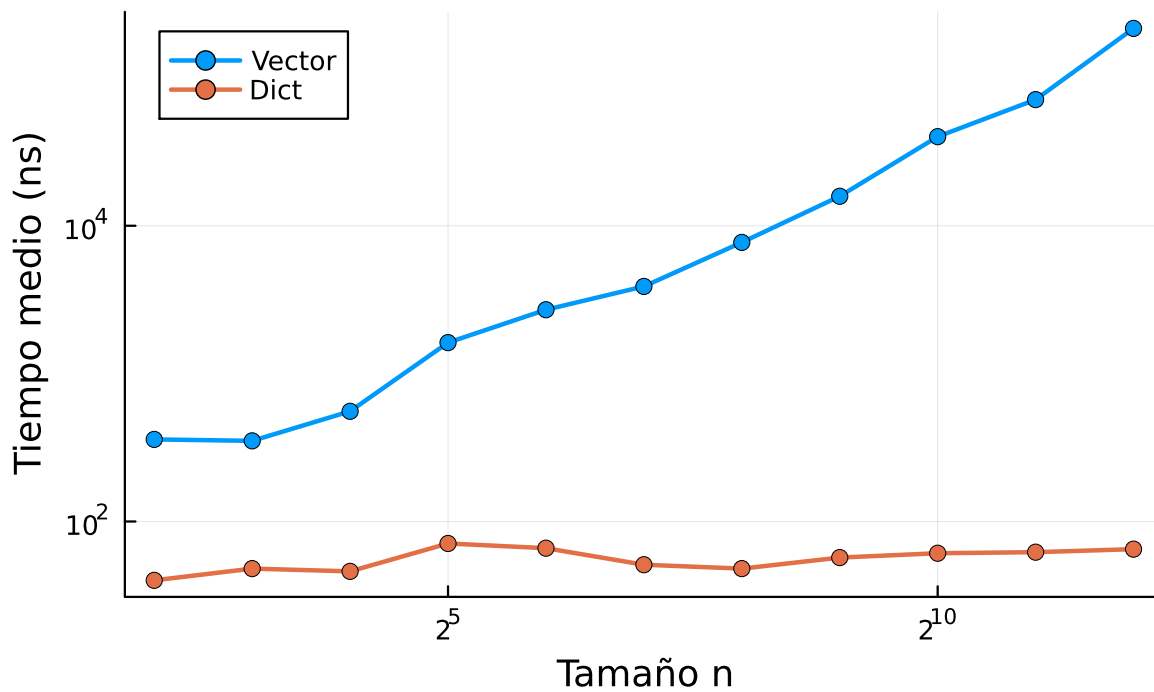
	n	vector_minimo	vector_medio	vector_maximo	dict_minimo	dict_medio	dict_maximo
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	4	200.0	359.0	5300.0	0.001	40.0006	100.0
2	8	300.0	351.0	4000.0	0.001	48.0005	100.0
3	16	500.0	556.0	4400.0	0.001	46.0005	100.0
4	32	1500.0	1620.0	9600.0	0.001	71.0003	100.0
5	64	2400.0	2704.0	8700.0	0.001	66.0003	100.0
6	128	3600.0	3887.0	15300.0	0.001	51.0005	100.0
7	256	7200.0	7717.0	19300.0	0.001	48.0005	100.0
8	512	14800.0	15826.0	35000.0	0.001	57.0004	100.0
9	1024	35000.0	40042.0	89000.0	0.001	61.0004	200.0
10	2048	64200.0	71276.0	128000.0	0.001	62.0004	300.0
11	4096	141400.0	216086.0	431800.0	0.001	65.0004	300.0

```

plot(
    tabla_insertar.n,
    [tabla_insertar.vector_medio tabla_insertar.dict_medio],
    label=["Vector" "Dict"],
    xlabel="Tamaño n",
    ylabel="Tiempo medio (ns)",
    title="Tiempo de insertar!",
    marker=:circle,
    linewidth=2,
    xscale=:log2,
    yscale=:log10
)

```

Tiempo de insertar!



```
for fila in eachrow(tabla_insertar)
    println("n=$(fila.n): gana $(fila.comparar)")
end
```

```
n=4: gana Dict
n=8: gana Dict
n=16: gana Dict
n=32: gana Dict
n=64: gana Dict
n=128: gana Dict
n=256: gana Dict
n=512: gana Dict
n=1024: gana Dict
n=2048: gana Dict
n=4096: gana Dict
```

2.4.2 Análisis de insertar!

Dict vuelve a ser la estructura más rápida para todos los tamaños de entrada probados. Sin embargo, **insertar!** es la operación donde el vector muestra el peor desempeño y también el

crecimiento más pronunciado en el tiempo medio conforme aumenta n . Esto se debe a que la inserción en un vector requiere recorrer todos los elementos hasta encontrar la posición o en otro caso una posición vacía, mientras que el diccionario utiliza una tabla hash que permite insertar directamente el nuevo par llave-valor.

3 Comparación de rangos

Usando la columna `comparar` de cada tabla determinamos el menor tiempo medio en cada tamaño.

```
function mostrar_rangos(tabla)
    tamaños_vector = [fila.n for fila in eachrow(tabla) if fila.comparar == "Vector"]
    tamaños_dict = [fila.n for fila in eachrow(tabla) if fila.comparar == "Dict"]

    println("Vector: ", tamaños_vector)
    println("Dict: ", tamaños_dict)
end

println("Buscar")
mostrar_rangos(tabla_buscar)

println("Borrar")
mostrar_rangos(tabla_borrar)

println("Insertar")
mostrar_rangos(tabla_insertar)
```

```
Buscar
Vector: Any[]
Dict: [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]
Borrar
Vector: Any[]
Dict: [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]
Insertar
Vector: Any[]
Dict: [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]
```

4 Discusión

Los tres experimentos son consistentes entre sí y con lo esperado teóricamente: para el rango de tamaños probado (n entre 4 y 4096), `Dict` fue más rápido que el vector en el 100 % de los casos, en las tres operaciones evaluadas. Esto concuerda con el análisis asintótico habitual de estas estructuras, donde el vector sin ordenar tiene un costo $O(n)$ para **buscar**, **insertar!** y **borrar!** (por el recorrido lineal necesario), mientras que la tabla hash del `Dict` tiene un costo esperado $O(1)$ para esas mismas operaciones, debido al costo del cálculo del hash y la eventual resolución de colisiones (Cormen et al., 2022).

Localidad de referencia. El vector que almacena los pares `Pair{Int64,Float64}` de manera que es posible recorrerlos de forma ordenada. Aunque esto puede representar una ventaja, en los experimentos realizados se observó que, al aumentar la cantidad de elementos, el tiempo necesario para recorrerlos también aumentó y esta ventaja no fue suficiente para compensar dicho costo.

En cambio `Dict`, distribuye sus entradas en función del valor hash de la llave, por lo que el acceso a una entrada concreta implica calcular dicho hash y casi directamente a la posición correspondiente en la tabla interna. Esto elimina la dependencia con n en el número de comparaciones, a costa de un patrón de acceso a memoria menos predecible que el de un recorrido lineal, pero que en promedio en este caso fue es más eficiente.

Costo de hash y colisiones vs. costo de recorrido. El experimento muestra que, incluso para los tamaños más pequeños evaluados ($n = 4$), el costo fijo de calcular el hash y acceder a la tabla del `Dict` (unos cuantos nanosegundos) ya es menor que el costo de recorrer linealmente 2 o 3 elementos de un vector. Esto sugiere que, para las llaves de tipo `Int64` usadas en esta actividad (cuyo hash es muy barato de calcular), la ventaja del `Dict` se manifiesta desde tamaños de entrada muy pequeños, y no solo para n grandes.

5 Conclusiones

En esta actividad se implementaron y compararon experimentalmente dos estructuras de datos: un vector de pares sin ordenar con capacidad fija, y un `Dict{Int64,Float64}` nativo de Julia. Se midieron los tiempos de las operaciones **buscar**, **insertar!** y **borrar!** para tamaños de entrada entre $2^2 = 4$ y $2^{12} = 4096$.

Los resultados muestran:

- `Dict` fue más rápido que el vector en las tres operaciones y para todos los tamaños de entrada evaluados. En ningún rango de n , dentro de lo probado el vector fuera preferible.
- El tiempo medio del vector crece con n en las tres operaciones, siendo especialmente pronunciado en **insertar!**.

- El tiempo medio del `Dict` se mantiene prácticamente constante frente a `n` en las tres operaciones.
- La ventaja del `Dict` se observa incluso para los tamaños de entrada más pequeños evaluados ($n = 4$)

Estas simulaciones confirman que la selección de algoritmos eficientes y estructuras de datos adecuadas es crucial para el desempeño de los programas, incluso en casos donde los tamaños de entrada son relativamente pequeños.

6 Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (2nd ed.). MIT Press.

JuliaLang. (s. f.). Collections and data structures. The Julia Language. <https://docs.julialang.org/en/v1/base/collections/>

JuliaCI. (s. f.). BenchmarkTools.jl: Manual. <https://juliaci.github.io/BenchmarkTools.jl/stable/manual/>

Téllez, E. S. (s. f.). Introducción al análisis de algoritmos con Julia: Capítulo 2 — Análisis de Algoritmos con Julia. INFOTEC. <https://sadit.github.io/ALGO-IR/cap2-analisis.html>